

Business Analytics using Python

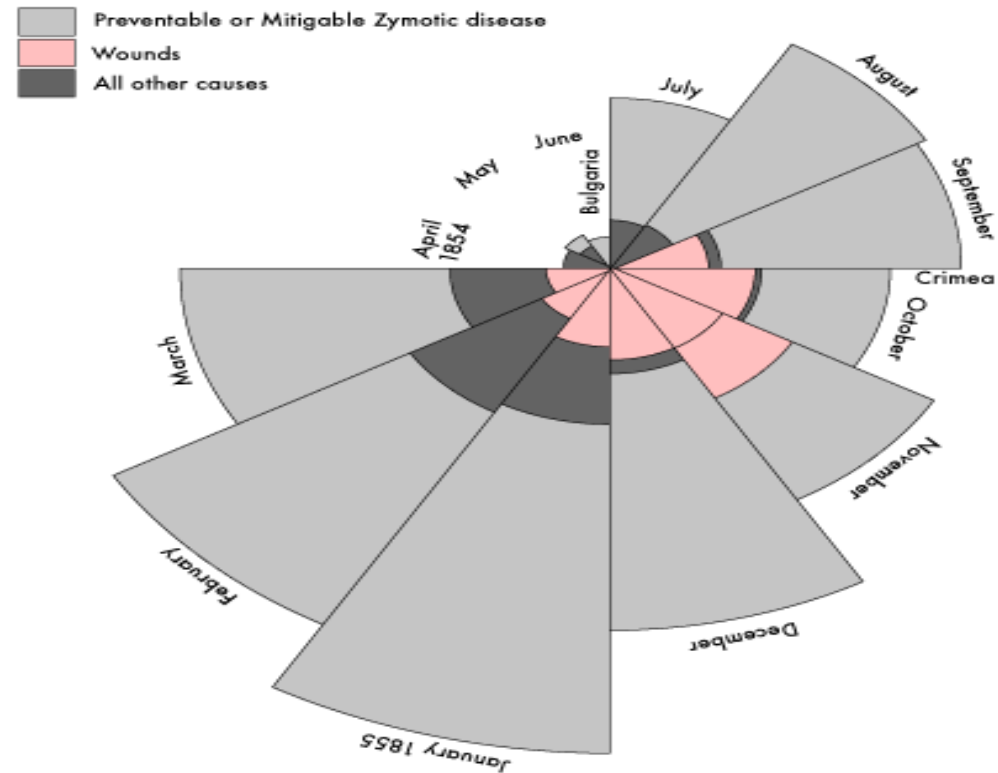
Nikhil D Jonathan



“To understand God’s thoughts we must study statistics, for these are the measure of His purpose.”

- Florence Nightingale ‘Lady with the Lamp’

Diagram of the Causes of Mortality in the Army in the East



The black line across November 1854 marks the boundary of the deaths from all other causes during that month. In October 1854, the black coincides with the red.

Florence Nightingale
1856

Why Analytics?

The inability of human beings to take decisions.

MONTY HALL PROBLEM (Savant, 1990)



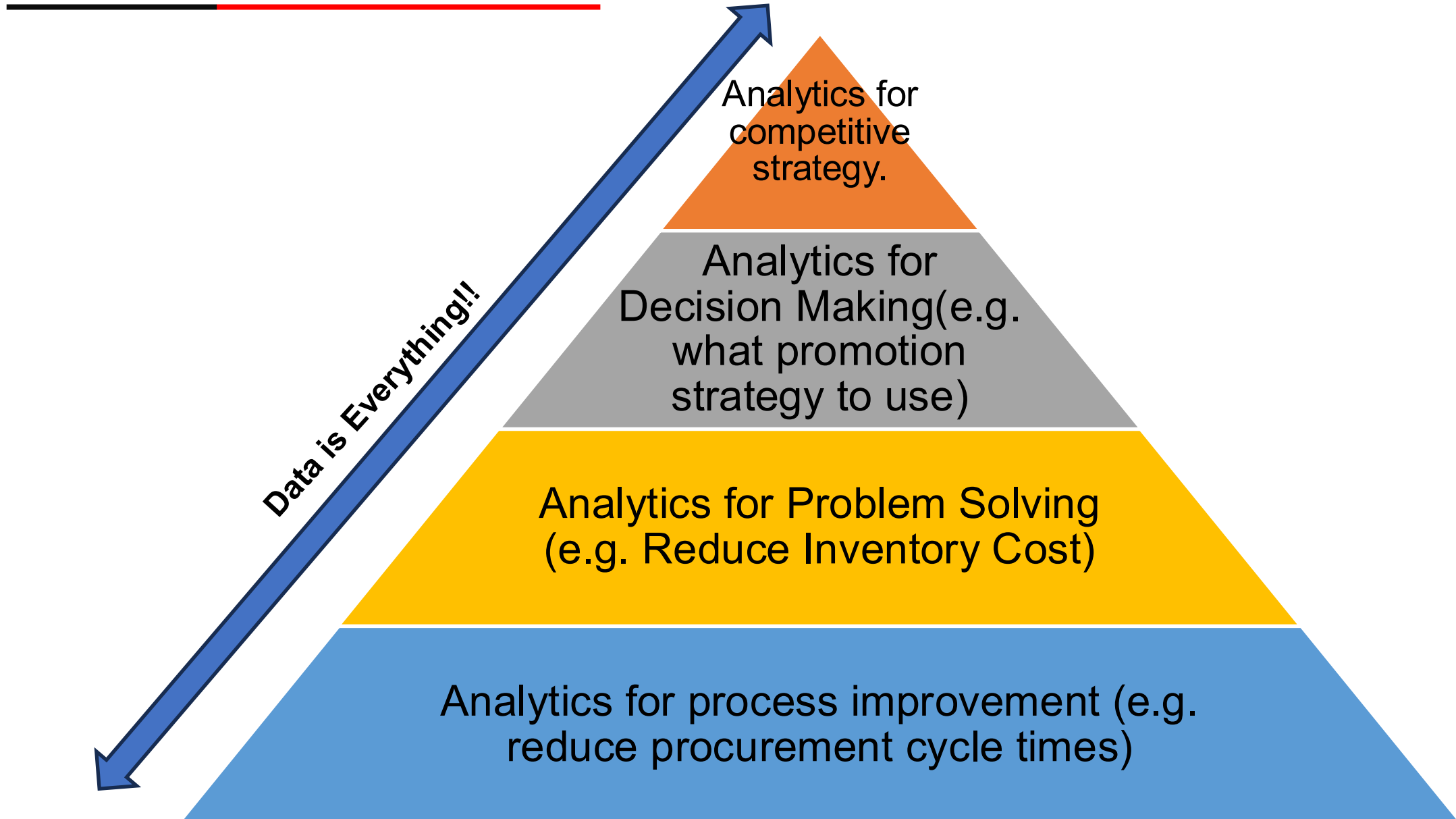
Behind door 1	Behind door 2	Behind door 3	Result if staying at door 1	Result if switching to the door offered
Car	Goat	Goat	Wins car	Wins goat
Goat	Car	Goat	Wins goat	Wins car
Goat	Goat	Car	Wins goat	Wins car

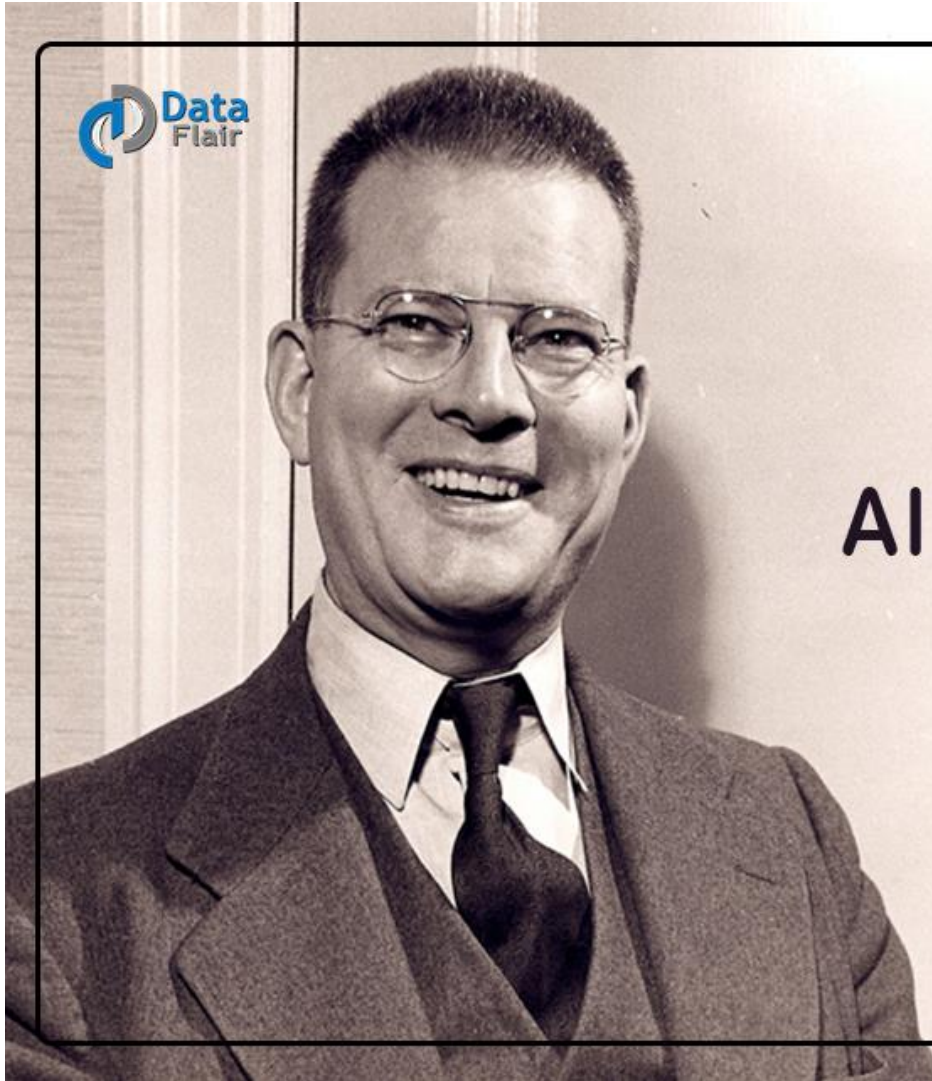
$1/3 = 33.33\%$

$2/3 = 66.66\%$

Why Business Analytics?

Maturity Model of Analytics





**In God we trust.
All others must bring data**

- W. Edwards Deming

**Statistician, Professor, Author,
Lecturer, and Consultant**

Another Why?

Placement/Career Progression Perspective

Reference:

1. <https://public.tableau.com/app/profile/world.economic.forum.data.analytics/viz/Futureofjobs-skillsonterise/skillsonterise>
2. <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>

Methods of Business Analytics:

1. DESCRIPTIVE ANALYTICS: Understand past performance
2. DIAGNOSTIC ANALYTICS: Discovering why events occurred
3. PREDICTIVE ANALYTICS: Forecasting Future Trends
4. PRESCRIPTIVE ANALYTICS: Recommending Optimal Actions

Peak Break-Up Times

According to Facebook status updates



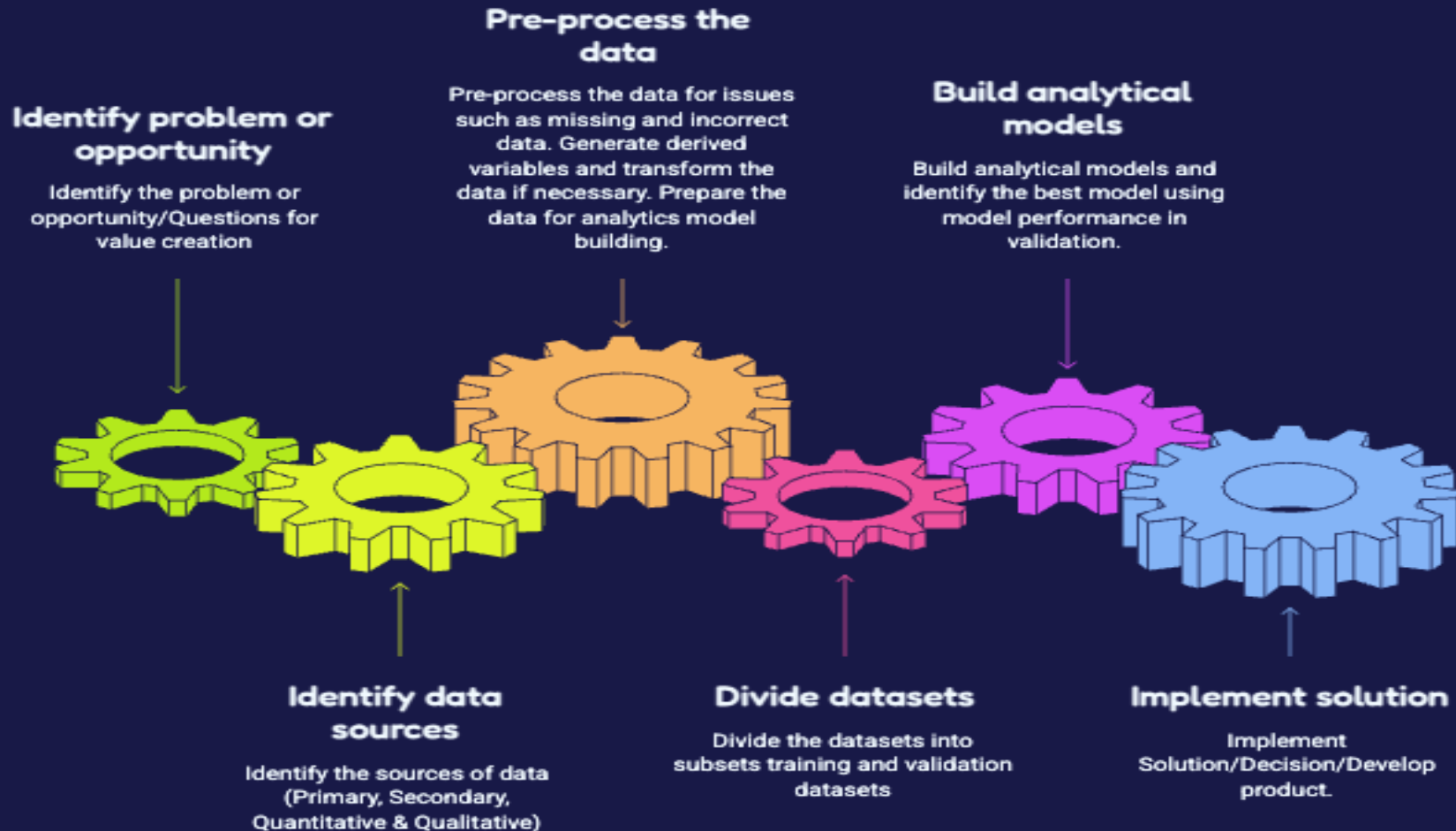
David McCandless & Lee Byron
InformationIsBeautiful.net / LeeByron.com

source: searches for "we broke up because"
taken from the infographic ultrabook
The Visual Miscellane um

Technology for practical Analytics (Data Warehousing):

1. Artificial Intelligence - “Agentic Systems, NLP, Transformers”
2. Machine Learning – “Supervised and Unsupervised”
3. Statistical Learning – “Linear and Non-linear”
4. Deep Learning – “ANN, CNN, RNN, LSTM & Reinforcement”

Stages in Data-Driven Business Analytics



What to expect from the business analytics course??

Skills:

Programming in Python;

Data Processing;

Data Visualizations;

Exploratory Data Analysis;

Statistical Inference & Testing;

Predictive Modelling; ML Fundamentals ;

CRISP-DM;

Data Story-telling; Responsible Analytics;

Let's make contact with Python

Why Python?

IDE – Kaggle, Colab Research & Anaconda

Working with Python

Analytical thinking remains the most sought-after core skill among employers, with **seven out of ten companies** considering it essential in 2025, and AI and big data top the list as the fastest-growing skills. (WEF, *Future of Jobs Report 2025*).

Python is how you *do* analytical thinking on data.

Python is also super-popular among programmers,

Ref: “The Importance of being earnest(TIOBE) index”

<https://www.tiobe.com/tiobe-index/>

History of Python

Python is one of the most popular, high-level, general-purpose programming languages. It was created with a focus on code readability and allows developers to write concepts in **fewer lines compared to C++ or Java**.

Creator: Guido van Rossum

Started: 1989, at Centrum Wiskunde & Informatica (CWI), Netherlands

Named after the BBC comedy series “Monty Python’s Flying Circus”



Integrated Development Environment (IDE) & Notebooks for business analytics:

They are a software tool that allows analysts to write, test, visualize and debug code, in a single narrative document.

Examples: Google Colab, Jupyter Notebooks, Databricks, Anaconda IDE, VS Code, Kaggle and Github Codespaces.

Feature	IDE (VS Code, PyCharm, etc.)	Notebooks (Jupyter, Databricks, Colab)
Execution	Runs the whole file sequentially	Runs code blocks individually
Primary Use	Building apps, APIs, production pipelines	Data analysis, visualization and experimenting
State	Resets every time you run the script	Keeps variables in memory between cells
Code Size	Excellent for massive, multi-file projects	Best for short, linear, single-file scripts
Testing	Built-in unit testing frameworks	Manual, visual inspection of outputs

Declaring Variables

A **variable** is a labelled box that stores a value so you can reuse it by name.

- You **declare** a variable with `=` → `name = value`
- The label is on the **left**, the value is on the **right**.
- Python figures out the **type** for you (no need to declare it).

Type	Meaning	Example
int	whole number	units_sold = 250
float	decimal number	price = 49.99
str	text (in quotes)	region = "South"
bool	True / False	is_premium = True

Operator	Syntax	Purpose	Example	Output
Addition	+	Sums values	9 + 3	12
Subtraction	-	Subtracts values	9 - 3	6
Multiplication	*	Multiplies values	9 * 3	27
Float Division	/	Divides to float	9 / 3	3.0
Floor Division	//	Divides to integer	9 // 2	4
Modulus	%	Finds remainder	9 % 2	1
Exponentiation	**	Raises to power	9 ** 3	729
Root (Log Inverse)	** (1/n)	Finds the n-th root	9 ** (1/2)	3.0

Conditional (Boolean) Variables

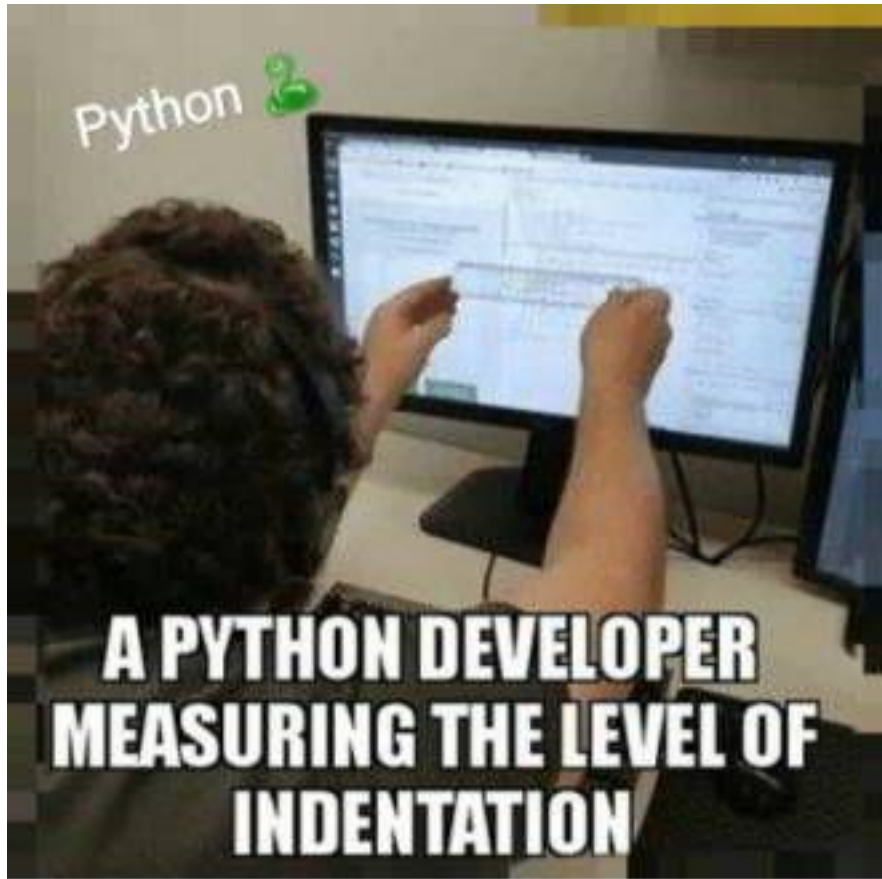
A **Boolean variable** holds only one of two values: `True` or `False`. They usually come from a **comparison** — a question Python answers with yes/no. The result of a comparison is a Boolean value you can **store in a variable** and reuse.

Operator	Question it asks
<code>==</code>	is equal to?
<code>!=</code>	is not equal to?
<code>> <</code>	greater / less than?
<code>>= <=</code>	greater-or-equal / less-or-equal?
<code>and</code>	are both true?
<code>or</code>	is at least one true?
<code>not</code>	flip true ↔ false

programmer joke:

!false

It's funny
because it's true



Indentation checking in Python is like:

Control Flow Statements

Control flow lets your code **make decisions** and **repeat work** — the core of any business rule.

(a) **if / elif / else** — take a different path depending on a condition.

(b) **for loop** — do something once for each item in a group.

(c) **while loop** — keep going until a condition stops being true.

⚠ Indentation (the spaces) is not decoration — it is the grammar. The indented lines are the ones that belong "inside" the decision or loop.

Collections

A **collection** holds many values in one variable. The four you will use most:
 The **dictionary** is the most important for business data — it is a labelled record.

Collection	Looks like	Use it when...	Changeable?
List	[10, 20, 30]	ordered items you may edit	Yes
Tuple	(19.0, 72.8)	fixed pair/record that must not change	No
Dictionary	{"key": value}	look up a value by a label	Yes
Set	{"A", "B"}	unique items, no duplicates	Yes

Functions

A **function** is a reusable recipe.

You write the steps **once**, give it a name, and call it whenever you need it — no copy-paste.

- `def` starts the recipe.
- Inputs go in the brackets — these are **parameters**.
- `return` hands back the answer.

Why it matters: functions stop you repeating yourself and make analysis auditable — everyone runs the *same* logic.

Functional Programming

A style where you **apply a function across a whole collection at once**, instead of writing a loop by hand. It is shorter and closer to how data tools (like `pandas`) actually work.

Three tools:

- **lambda** — a tiny one-line unnamed function.
- **map** — apply a function to *every* item.
- **filter** — keep only the items that pass a test.
- **List comprehension** — the readable Python way to do both.

Modules & Packages

You never build analytics from scratch. A **module/package** is a ready-made toolbox someone else wrote that you `import`.

- `import math` — a built-in toolbox.
- `import pandas as pd` — the industry toolbox for data tables (a **package**). `as pd` gives it a short nickname.

This is the bridge to the rest of the course:

pandas for data

Matplotlib & seaborn for charts

scikit-learn for machine learning — all imported this same way.

“This was one of the best parts of being a coder, and an artist: the thrill of being in the middle of creating something delightful. It’s like the anticipation of eating freshly baked bread after its aroma fills the room.”

– Dr. Joy Buolamwini, *Unmasking AI*

“The code you write makes you a programmer. The code you delete makes you a good one. The code you don’t have to write makes you a great one.” – Mario Fusco, Principal Software Engineer at Red Hat

“The function of good software is to make the complex appear to be simple.” – Grady Booch, Chief Scientist for Software Engineering at IBM Research

“Code is read much more often than it is written.” – Guido Van Rossum, creator of Python

“Learning to write programs stretches your mind, and helps you think better, creates a way of thinking about things that I think is helpful in all domains.” – Bill Gates, Microsoft co-founder

Project Work – Teams – Tracks – Evaluation Pattern

Mid-Term – Case-Studies & Questions (1)

End-Term – Case Studies & Questions (3)

Classroom Activities & Assignment – Case-Studies and Questions (3)